

RETO TÉCNICO: MODERNIZACIÓN ARQUITECTÓNICA DE UN SISTEMA CORPORATIVO HEREDADO (SOLID)

DOCUMENTO ÚNICO DE ENTREGABLES (FASES 0 A 5)
Sistema: Hacienda

IDENTIFICACIÓN DEL EQUIPO Y ROLES DE TRABAJO

Integrante (Nombre)	ID	Rol Asignado	Frente de Responsabilidad	% Part.
Integrante 1: Diego Villegas	000553180	Arq. de Dominio	Identificación de responsabilidades/límites de cada clase (SRP), modelo del dominio, jerarquías de herencia y validez (LSP).	100%
Integrante 2: Salome Fonseca	000248599	Arq. de Dependencias	Mapa de dependencias, abstracciones (interfaces), inversión e inyección de dependencias, composition root (DIP, ISP).	100%
Integrante 3:	000543839	Ing. de Comportamiento	Pruebas de caracterización,	100%

María Fernanda Muñoz			evidencia de preservación de conducta observable, escenarios de ejecución del programa principal.	
Integrante 4: Pilar Mantilla	000547259	Integrador y Evidencia	Consistencia diagrama-código, estructura del entregable, bitácora de uso de IA, métricas antes/después.	100%

TABLA DE CONTENIDO

FASE 0 — LECTURA EN FRÍO (ANÁLISIS PREVIO INDIVIDUAL)	4
Lectura en Frío - Integrante 1: Diego Alejandro Villegas	4
Entidades principales del sistema:	4
Lectura en Frío - Integrante 2: Maria Fernanda Muñoz Acosta	6
Lectura en Frío - Integrante 3: Salome Fonseca	7
Lectura en Frío - Integrante 4: Maria del Pilar Mantilla Gelves	10
FASE 1 — DIAGNÓSTICO DEL SISTEMA ACTUAL (AS-IS)	15
1.1 Diagrama UML del Estado Actual (AS-IS)	15
1.2 Inventario de Hallazgos en el Código Heredado	15
1.3 Mapa de Dependencias Actual	19
1.4 Cuatro Puntos de Dolor Priorizados	22
FASE 2 — LOS CAMBIOS QUE VIENEN (ANÁLISIS DE IMPACTO AS-IS)	24
2.1 Catálogo de Solicitudes de Cambio	24
2.2 Matriz de Impacto en Arquitectura Actual (AS-IS)	24
FASE 3 — DISEÑO DE LA NUEVA ARQUITECTURA (TO-BE)	27
3.1 Diagrama UML de la Nueva Arquitectura (TO-BE)	27
3.2 Justificación de la Aplicación de los 5 Principios SOLID	27
Single Responsibility Principle (SRP)	27
Open/Closed Principle (OCP)	28
Liskov Substitution Principle (LSP)	29
Tabla de Verificación de Sustituibilidad (LSP)	30
Interface Segregation Principle (ISP)	31
Dependency Inversion Principle (DIP)	1
3.3 Registros de Decisión Arquitectónica (ADR)	1
ADR-01: [so de Herencia para los nuevos productos (SC-H1)]	1
ADR-02: [Reducción de responsabilidades de la clase Hacienda]	1
ADR-03: [Inversión de dependencias en la persistencia de Productos]	1
ADR-04: [Refactorización y aislamiento de reglas en la clase Potrero]	1
ADR-05: [Título Corto de la Decisión - ej. Implementación de Repositorios para Datos]	1
FASE 4 — IMPLEMENTACIÓN Y EVIDENCIA	1
4.1 Código Fuente y Repositorio	1
4.2 Evidencia de Preservación del Comportamiento	1
FOTO DE EVIDENCIA	1
4.3 Prueba de Extensibilidad y Métricas Comparativas	1
4.4 Bitácora de Uso de Inteligencia Artificial (IA)	1
FASE 5 — PLANIFICACIÓN DE LA SUSTENTACIÓN EN VIDEO	1
5.1 Enlace del Video de Sustentación	1
5.2 Estructura y Planificación de Tiempos del Video (Máx. 20 min)	1

FASE 0 — LECTURA EN FRÍO (ANÁLISIS PREVIO INDIVIDUAL)

Lectura en Frío - Integrante 1: Diego Alejandro Villegas

1. ¿Qué cree que hace el sistema y cuáles son sus entidades principales?

El sistema básicamente lo que hace es manejar una hacienda de ganadería, en donde principalmente se enfocan en tenerle como una trazabilidad a tres tipos de ganados, que son: Cebón, Novillo y el Ternero, ya que la hacienda ganadera se dedica a vender este tipo de ganados.

Además, también se maneja la parte de tener un control con la enfermedad Bacteriana, esta es una enfermedad que es causada por Bacterias.

Entidades principales del sistema:

1. **Res:** Esta entidad resumidamente será la plantilla o la base para que las otras reses (Cebón, Novillo, Ternero), se construyan, ya que tienen características en común, pero hay acciones que son diferentes.
2. **Potrero:** Se encarga de alojar la res principalmente, aunque teniendo en cuenta las condiciones de cada una.
3. **Hacienda:** Esta es la principal de todas, ya que tiene todas las acciones que hace el sistema, desde añadir res a un potrero, vender, alimentar, etc.
4. **Vacuna:** Es la base para tener la trazabilidad de todas las

	vacunas que se tengan que aplicar a las reses
2. Los tres lugares del código que harían costoso un cambio, y por qué.	A. En muchas partes del código se aplican demasiadas condiciones, una tras otras, esto implica que el código sea de pronto un poco confuso, difícil de mantener, que gaste más tiempo y memoria evaluando tantas reglas y opciones. B. Esta también va muy de la mano con la primera, pero por poner un ejemplo, a la hora de añadir una res a un potrero, hay un switch ineficiente, cuando hacemos un cambio en un tipo de potrero hay que tocar el código fuente, lo que viola el principio solid open/closed. C. También si observamos que la autenticación es demasiado débil, si mañana queremos agregar un nuevo usuario que tenga privilegios en la aplicación, toca modificar el código fuente.
3. Una pregunta que le haría al programador original si pudiera.	1. ¿Por qué tiene absolutamente todas las acciones que tiene que hacer el sistema en una sola entidad? si mañana hay una acción nueva toca tocar el código fuente 2. Por qué las reglas son constantes, que pasa si las reglas cambian después, ¿Cómo harías ese cambio?

Lectura en Frío - Integrante 2: Maria Fernanda Muñoz Acosta

1. ¿Qué cree que hace el sistema y cuáles son sus entidades principales?	El sistema es de una hacienda, la cual, tiene varios tipos de reses, clasificados según sus edades, los cuales pueden ser añadidos a varios potreros. Dentro del sistemas, dependiendo del rol que se tenga como usuario, se puede crear potreros, agregar reses, crear vacunas, entre otras cosas. A las reses se les puede alimentar, vender, vacunar, etc. Entidades principales: Si bien, todas las clases son importantes, porque igualmente a la mayoría toca hacerle cambios, considero que las principales son: Clase hacienda Clase potrero Clase res Clase vacuna Clase autenticación
2. Los tres lugares del código que harían costoso un cambio, y por qué.	A. Cualquier cambio que se le tenga que hacer a la clase 'Hacienda', debido a que es una clase la cual cuenta con muchísimos métodos, los cuales no están respetando principalmente el principio SRP, por lo que para mejorar el código, se le tienen que hacer muchísimos cambios al diseño de la arquitectura. B. Lo mismo sucede con la clase 'Potrero', ya que además de romper con el principio SRP, también está faltando al principio OCP, ya que al no usar interfaces, por ejemplo para el caso de definir rangos de edad según el tipo de potrero, al momento que se quiera agregar uno, toca modificar el código ya existente. C. En la clase 'Autenticacion', también cuenta con un problema de SRP y en caso de que se quieran crear más tipos de roles también toca modificar el código, aumentando así ciclos innecesarios. Además el hecho de agregarle más validaciones, permisos y

	roles, lo volvería costoso porque impacta con la seguridad.
3. Una pregunta que le haría al programador original si pudiera.	¿Por qué en una clase como 'hacienda', se decidió por colocar varios métodos que de por sí tienen que ver mucho más con otras clases que con esa misma? Aunque las fuera a colocar con un diseño incorrecto, por lo menos varias quedarían mucho más "coherentes" en otras clases.

Lectura en Frío - Integrante 3: Salome Fonseca

1. ¿Qué cree que hace el sistema y cuáles son sus entidades principales?	A partir de un analisis rapido del codigo se aprecia que el esta aplicacion de gestion ganadera controla lo que es: Poteros, reses dentro de esos potreros, vacunas, usuarios y autenticaciones, ventas de animales Sus entidades principales son: - Hacienda.cs: Esta se encarga de la logica global y mantiene listas de potreros, ventas y vacunas, esta tambien coordina aca lo que es crear, buscar potreros, agregar reses a un potrero, vender una res, alimentar una res, crear vacinas y registrar su aplicacion - Potrero.cs Esta representa un espacio fisico de la hacienda donde viven las reses, tiene una identificacion, un tipo de potrero (ternero, cebon, novillo) y una lista de las reses que contiene, su responsabilidad principal es validar si una res puede entrar segun su edad y su tipo de potrero y crea una instancia correcta de res segun la categoria - Res.cs: Es la clase principal de la que partes todos los animales de la hacienda. No representa lo que es un animal especifico, sino que reúne las
--	--

características que todos ellos tienen en común, cuanta con atributos básicos como:

- Nombre
- Peso
- Edad
- L_vacunas_aplicadas

A partir de esta clase se crean las demás especialización como: ternero, cebón y novillo, el propósito es que todas las reses compartan la misma información básica, pero que cada una pueda tener sus propias características según la etapa o categoría a la que pertenecen

- **Cebon.cs\Ternero.cs\Novillo.cs**

Lo que es ternero, cebón y novillo son especializaciones de la clase Res que representan las diferentes etapas de crecimiento de las reses, en ternero representa lo que es una res joven de temprana edad y se usa clasificando el animal según su edad y el tipo de potrero donde se ubica, en cebón se clasifica como una res en etapa intermedia de crecimiento y se usa clasificando reses que exceden la edad de un ternero pero que todavía no alcanzan la categoría de novillo usando la regla de ReglaRes.edad_max ubicada en potrero donde especifica el mínimo y máximo de edad de cada res, en la clase de novillo es la categoría más madura de las reses del dominio en donde tiene 49 en adelante, se usa para representar animales en la etapa final de crecimiento

- **Vacuna.cs**

Representa el catálogo de vacunas disponibles de la hacienda, tiene datos como, nombre, lote, fecha de vencimiento, fecha de aplicación, periodo de aplicación, su rol en el código es modelar el tratamiento aplicado al ganado y permitir registrar que vacunas se aplicaron

	● Venta.cs Es el registro histórico de una transacción comercial en esta se guardan la relación entre el potero donde estaba la res, la res vendida, la fecha de venta y el monto pagado. ● Usuario.cs En esta representa a la persona que puede ingresar al sistema y realizar acciones segun el rol que tenga cuenta con atributos como, nombre, contraseña y se utiliza para validar el acceso al sistema y controlar los permisos
2. Los tres lugares del código que harían costoso un cambio, y por qué.	A. Hacienda.cs Es una clase central en donde se está coordinando múltiples operaciones diferentes, si se cambia una regla del negocio o se agrega nuevas operaciones es muy probable que haya que tocar esta misma clase y sería más probable que llegara a quebrarse B. Potero.cs Combina validaciones, decisiones de creación de animales segun el tipo de potero y disparo de eventos, eso la vuelve costosa de extender porque cada cambio de regla obliga a revisar varios comportamientos al mismo tiempo, también volviéndola más propensa a quebrarse C. Autenticacion.cs Tiene una lógica de permisos rígida no pensada para extenderse basada en nombre de usuario como admin, empleado y visitante, si se agregaran nuevos roles o nuevas políticas de autorización la clase probablemente requiera cambios directos tomando el

	riesgo de que se quiebre o deje de funcionar con un simple cambio
3. Una pregunta que le haría al programador original si pudiera.	¿Por qué decidieron implementar toda la lógica del programa (reglas, estados, eventos, etc.) en una sola clase, como Hacienda, en lugar de separar la lógica de cada proceso en clases o componentes diferentes?

Lectura en Frío - Integrante 4: Maria del Pilar Mantilla Gelves

1. ¿Qué cree que hace el sistema y cuáles son sus entidades principales?	• Estructura de clases: • Clase venta: tiene potrero (de la clase potrero), fecha, res y monto. • Clase Vacuna: Clase abstracta, clase padre. Esta clase tiene nombre, lote, fecha vencimiento y fecha aplicación. • Clase Bacteriana --> Hereda de la clase Vacuna, entonces Vacuna (clase padre), tiene una excepcion de validar que el periodo de aplicación este entre ciertas semanas. • Clase Viva --> Hereda de la clase Vacuna, entonces Vacuna (clase padre), esta clase tiene enum para atenuaciones. • Clase Usuario: Esta clase tiene nombre y contraseña. • Clase Res: Clase abstracta, clase padre. Tiene nombre, peso, edad y la lista de
--	--

	vacunas aplicadas (Clase padre Vacuna) ● Clase ternero --> Hereda de la clase Res (Clase padre), esta clase tiene nombre, peso y edad. Tiene una excepción en edad si excedió la edad máxima. ● Clase Novillo --> Hereda de la clase Res (Clase padre). Tiene una excepción en edad si excedió la edad máxima. ● Clase Cebón --> Hereda de la clase Res (Clase padre). Tiene una excepción en edad si excedió la edad máxima. ● Clase Autenticacion: Esta clase implementa una interfaz IAutenticacion, esta interfaz contiene un método de autorizar operación del usuario. La clase tiene una lista de usuarios registrados, en el constructor crea usuarios por defecto. Contiene un método para crear usuarios, método para listar usuarios, método para validar credenciales, método para buscar usuarios por nombre, método para autorizar operación ● Clase Potrero: La clase tiene enum de tipos de potreros, identificación, lista de reses y una lista
--	---

	tipo de potrero. Esta clase esta suscrito a varios eventos: potrero mitad, potrero lleno, peso venta, peso min. Contiene un método para añadir las reses al potrero. Este método tiene excepciones: valida nombre no vacío, valida potrero lleno. Contiene switch cases para definir rangos de edad según el tipo de potrero y otro para el tipo de vaca. Se suscribe a los eventos y dispara los eventos. Mensaje final de retorno de que han sido añadidas las reses al potrero. Existe otro método para buscar la res por el nombre. ● Clase Hacienda: Implementa las interfaces IVacunacion, IVentaRes, ICreacionVacuna, esta clase contiene lista potreros, ventas y vacunas. Métodos de crear potrero, buscar potrero, añadir res al potrero, vender res, alimentar res, sobrecarga metodo alimentar res, crear vacuna al inventario, crear vacuna viva individual, crear lote de vacunas bacterianas, crear lote de vacunas vivas, aplicar vacunas.
--	---

	Qué hace el sistema: El sistema se trata sobre la gestión de la hacienda ganadera, controla potrero, reses (ternero, novillo, cebon), vacunación, ventas y usuarios. En el sistema se clasifica el ganado por edad, valida la capacidad del potrero, etc Clases principales: Clase Res, clase Potrero, clase Hacienda, clase Vacuna.
2. Los tres lugares del código que harían costoso un cambio, y por qué.	A. Hacienda concentra mucha parte de las reglase de negocio, esta tan acoplada que cambiar algo aquí puede afectar muchas partes del codigo y de las reglas por lo que sería muy costoso su cambio. B. En la clase potrero hay muchos switch cases, entonces a la hora de agregar nuevos tipos se tendría que abrir de nuevo el código lo que hace que se corra el riesgo de que ocurran errores humanos. C. La clase de autenticación tienen tipos de usuarios quemados como admin, empleados y visitantes lo que hace que el código sea restrictivo.
3. Una pregunta que le haría al programador original si pudiera.	¿Por qué se concentró tantas reglas de negocio en la clase Hacienda?

FASE 1 — DIAGNÓSTICO DEL SISTEMA ACTUAL (AS-IS)

1.1 Diagrama UML del Estado Actual (AS-IS)

El archivo del diagrama del estado actual se adjunta en el repositorio.

1.2 Inventario de Hallazgos en el Código Heredado

La siguiente tabla documenta las deudas técnicas, malas prácticas y violaciones de principios de diseño identificadas en el sistema:

ID	Ubicación (clase/línea)	Síntoma Observado	Principio Comprometido	Impacto en Negocio (Costo/Riesgo/ Tiempo)	Severidad / Origen
H-01	Autenticacion .cs	La clase gestiona todo lo que tiene que ver con usuarios, válida autenticación, además de que también tramita la parte de autorizacion	SRP (Single Responsibility Principle)	Tiempo alto de cambio: Cualquier cambio en una de las tres áreas, tenemos que modificar el archivo existente, volver a probar, a compilar, y hacer todo este proceso Riesgo alto: Además de que demanda mucho tiempo, es una operación de alto riesgo, esto debido a que puede tumbar partes de el sistema	Alta/ [PROPIO]
H-02	Autenticacion .cs	El método de autorización se encarga de dar permisos según el	OCP (Open closed principle)	Riesgo alto: Tenemos un riesgo grande de seguridad	ALTO [PROPIO]

		rol, es ineficiente ya que si quiero agregar un rol nuevo en un futuro me toca modificar el código, lo cual sería una mala práctica de programación.		porque los datos se comparan como un texto plano, si alguien accede a una base de datos o a la memoria, fácilmente puede tener estos datos Costo alto: Agregar un nuevo rol implica abrir este método y modificarlo, con el riesgo de romper la lógica de los roles ya existentes	
H-03	Potrero.cs	Potrero se encuentra muy mal diseñada ya que vemos que esta clase hace muchas cosas, orquesta eventos, además de que hace una mezcla de reses, y la de validación de edad.	SRP Single Responsibility Principle	Costo alto: Porque cada vez que el negocio necesite un tipo de potrero nuevo o un cambio en las reglas de edad, alguien va a tener que meterse a modificar esta misma clase que también controla cómo se crean los animales y cómo se avisan los eventos	ALTO [Asistido]
H-04	Potrero.cs	Esta clase además, tiene un gran	OCP (Open Closed Principle)	Riesgo alto: Se considera un	ALTO [Propio]

		problema de diseño al tener fijos nada más tres tipos de res, aparte que maneja casos para agregar reses que son exclusivos nada más para esas 3 que hay.		riesgo alto debido a que si quisiera agregar una nueva res, se podría dañar la lógica existente de algo mas y tumbar esa parte de la aplicación	
H-05	Potrero.cs	Potrero que es la clase de alto nivel crea directamente instancias concretas de las reses, en lugar de depender de alguna abstracción que se encargue de esa creación.	DIP (Inversión de Dependencias)	Costo alto: Potrero está acoplado a estas clases, en caso de que alguna de esas clases cambie, la clase de alto nivel Potrero también se ve obligada a cambiar	ALTO [Propio]
H-06	Novillo.cs	Tiene su propia versión de poner la edad	LSP (Sustitución de Liskov)	Riesgo medio: Porque el error no se nota al programar ni al compilar aparece de sorpresa cuando el sistema ya está en uso, en el momento en que alguien le asigna una edad inválida a un Novillo	MEDIO [Propio]
H-07	Ternero.cs	Tiene su propia versión de poner la edad	LSP (Sustitución de Liskov)	Riesgo medio: Porque el error no se nota al programar ni al compilar aparece de	MEDIO [Propio]

				sorpres a cuando el sistema ya está en uso, en el momento en que alguien le asigna una edad inválida a un Ternero	
H-08	Cebon.cs	Tiene su propia versión de poner la edad	LSP (Sustitución de Liskov)	Riesgo medio: Porque el error no se nota al programar ni al compilar aparece de sorpresa cuando el sistema ya está en uso, en el momento en que alguien le asigna una edad inválida a un Cebon	MEDIO [Propio]
H-09	Hacienda.cs	Es una clase que contiene mucha lógica que no debería aplicar. Gestiona inventario de vacunas, ventas, creación de potreros, alimentación y aplicación de vacunas.	SRP (Responsabilida d Única)	Riesgo Alto: Cualquier modificación al sistema obliga a tocar esta clase, arriesgando romper funciones no relacionadas	[Alta] [Propio]
H-10	Hacienda.cs	En aplicar_vacuna, la hacienda hace comprobaciones de tipo explícitas: if (res is Ternero) {...} else if (res is Novillo) {...}.	OCP (Abierto/Cerrad o)	Tiempo Alto: Si entra un nuevo tipo de res al dominio del negocio, toca venir a modificar este condicional.	[Alta] [Asistido]

H-11	Hacienda.cs	La clase usa el operador new instanciando clases concretas como new Potrero(...), new Venta(...), new Bacteriana(...) y new Viva(...).	DIP (Inversión de Dependencias)	Costo Medio: Fuerte acoplamiento hacia los detalles (clases de bajo nivel). Dificulta enormemente aislar la clase Hacienda para hacerle pruebas unitarias.	[Media] [Propio]
H-12	Hacienda.cs	Reglas de negocio quemadas en el código. Por ejemplo: if (cantidad > 100) al crear lotes de vacunas.	OCP (Abierto/Cerrado)	Riesgo Medio: Si se decide que ahora se pueden comprar lotes de 200 vacunas, toca recompilar el código fuente en lugar de cambiar una simple variable de configuración.	[Media] [Propio]

1.3 Mapa de Dependencias Actual

Clases de Alto Nivel (Reglas del Negocio):

Entidades Principales y Agregados del Dominio (Modelos del Negocio):

- **Hacienda:** Agregado raíz que orquesta la finca ganadera (gestión de potreros, vacunas, ventas, usuarios, cálculo de consumo de alimento, reportes de salud y proyección de ganancias).
- **Potrero:** Entidad que representa la unidad de pastoreo/mantenimiento (gestiona aforo, adición/remoción de ganado y tipos de potrero: pastoreo, engorde, maternidad, cuarentena).
- **Res (Clase Abstracta):** Entidad base para el ganado (define el consumo de alimento, cálculo de desarrollo y aptitud para venta).
- **Ternero:** Subclase especializada de Res (reglas de nutrición inicial y etapa de desarrollo).

- **Novillo:** Subclase especializada de Res (reglas de crecimiento intermedio).
- **Cebon:** Subclase especializada de Res (reglas de ceba/engorde final previo al sacrificio).
- **Vacuna (Clase Abstracta):** Entidad base del plan sanitario (lote, fecha de aplicación, fecha de vencimiento y cálculo de vigencia EstaVencida()).
- **Bacteriana:** Subclase de Vacuna con reglas de periodo de re-aplicación.
- **Viva:** Subclase de Vacuna para cepas vivas atenuadas.
- **Venta:** Entidad de negocio que representa la transacción comercial de una res.
- **Usuario:** Entidad que modela los usuarios y roles del sistema.
- **Autenticacion:** Lógica de dominio para validación de credenciales.

Reglas del Negocio y Validadores del Dominio:

- **ReglaRes:** Lógica para validar límites de peso, edad y etapa productiva del ganado.
- **ReglaPotrero:** Lógica para validar capacidad máxima de potreros y densidad ganadera.
- **ReglaVacuna:** Lógica para controlar esquemas de vacunación y alerta de dosis vencidas.
- **Validacion / ValidarRes / ValidarPotrero / ValidarVacuna / ValidarVenta:** Clases de validación de negocio para mantener la integridad de los datos previo a cualquier transacción.

Eventos del Dominio (Publishers de Negocio):

- **PublisherPesoMin / PublisherPesoVenta:** Despachadores de eventos cuando un animal alcanza pesos clave.
- **PublisherPotreroLleno / PublisherPotreroMitad:** Eventos emitidos según la ocupación del potrero.
- **PublisherVacunacionComplelada / PublisherVacunaVencida:** Eventos del ciclo sanitario del ganado.

Clases de Bajo Nivel (Detalles Técnicos):

• Persistencia e Infraestructura de Almacenamiento:

- **PersistenciaService:** Servicio encargado del acceso a datos físico mediante lectura y escritura directa de archivos de texto plano (Potreros.txt, Reses.txt, Ventas.txt, Vacunas.txt, Usuarios.txt) en el directorio local Datos/.

• Servicios de Aplicación u Orquestación:

- **PotreroService, ResService, VacunaService, VentaService, UsuarioService:** Servicios intermedios que coordinan la interacción entre la interfaz web y PersistenciaService.

- **Capa de Presentación y UI (Controladores y DTOs MVC):**

- **PotreroController, ResController, VacunaController, VentaController, AccountController, UsuarioController, HomeController:** Controladores HTTP que reciben peticiones de la interfaz gráfica web y renderizan vistas en Razor.
- **LoginViewModel, ErrorViewModel:** Modelos de datos para entrada de formularios web y captura de errores de UI.

- **Frameworks Externos y Soporte Técnico (AOP / Middleware / Web):**

- **InterceptorAutenticacion:** Interceptor técnico basado en la librería externa Castle.DynamicProxy para control de ejecución.
- **InterceptorValidarInformacion:** Interceptor de validación acoplado a la infraestructura de ASP.NET Core (HttpContextAccessor) para inyectar errores en la sesión HTTP (HttpContext.Items).
- **Program.cs:** Punto de entrada de la aplicación técnica donde se configura el servidor Kestrel, inyección de dependencias concretas y middlewares web.

Inversión de Relación (Estado Actual):

- **Diagnóstico:** El sistema actual NO cumple con el Principio de Inversión de Dependencias (DIP). Presenta un acoplamiento rígido de alto nivel hacia bajo nivel, además de filtraciones de conceptos técnicos hacia la biblioteca de dominio.

- **Análisis Detallado:**

1. **Ausencia de Abstracciones (Interfaces de Persistencia y Repositorios):**

Tanto en los controladores MVC (ResController, PotreroController) como en Program.cs, la aplicación depende e inyecta directamente clases concretas (ej. `builder.Services.AddSingleton<PersistenciaService>()`). No existen interfaces como `IPersistenciaService` o `IResRepository`. Esto significa que el alto nivel y los servicios dependen explícitamente de la implementación concreta.

2. **Acoplamiento Directo al Almacenamiento en Disco:**

Los servicios y la persistencia dependen directamente de clases de bajo nivel del sistema operativo (System.IO.File, Path) para operar sobre archivos .txt. Cambiar la fuente de datos a una base de datos relacional (SQL Server/EF Core) requeriría modificar directamente el código de los servicios.

3. **Contaminación de Infraestructura Web en el Dominio (Fuga de Abstracción):**

Las clases de validación de negocio y sus interceptores (InterceptorValidarInformacion) dependen de librerías externas (Castle.DynamicProxy) y del contexto de red de ASP.NET Core (IHttpContextAccessor), depositando mensajes en HttpContext.Items. Esto acopla las reglas del negocio a una aplicación web HTTP, impidiendo ejecutarlas en pruebas unitarias puras o en otros entornos (consola, tareas en segundo plano, etc.).

• **Flujo de Dependencias Original: Cliente Web (Navegador)**

- El cliente interactúa con la interfaz web → ResController / PotreroController (Capa UI - Bajo Nivel).
- El controlador llama directamente a la implementación concreta ResService / PotreroService (Bajo Nivel).
- El servicio delega el almacenamiento a la clase concreta PersistenciaService (Bajo Nivel).
- PersistenciaService utiliza Castle.DynamicProxy e IHttpContextAccessor (Bajo Nivel) para ejecutar los validadores (ValidarRes, ValidarPotrero - Alto Nivel) y escribir el resultado directamente en archivos .txt en disco.

Marco Técnico / Framework: Castle.DynamicProxy e IHttpContextAccessor [Bajo Nivel]

Modelos de Negocio: Potrero, Res, ValidarRes, ValidarPotrero [Dominio - Alto Nivel]

1.4 Cuatro Puntos de Dolor Priorizados

1. Punto de Dolor Prioritario 1: [Vulnerabilidad de seguridad —Clase: Autenticación]

- **Trazabilidad a Hallazgo:** H-01, H-02
- **Criterio de Priorización:** Tener roles y contraseñas en texto plano impide auditar o cambiar credenciales sin recompilar la aplicación. Además, lanzar una excepción para confirmar que un usuario "está autorizado" destruye el rendimiento del sistema, consumiendo ciclos de CPU innecesarios y dificultando el control normal de la aplicación. Su corrección previene caídas inminentes y brechas de seguridad fatales.

2. Punto de Dolor Prioritario 2: [Acumulación descontrolada de notificaciones y bloqueos en potrero— Clase: Potrero]

- **Trazabilidad a Hallazgo:** H-03

- **Criterio de Priorización:** Cada vez que se ingresa un nuevo animal a un potrero, el sistema duplica y acumula notificaciones internamente en lugar de procesarlas de forma limpia. Con el uso diario y el aumento del ganado, la aplicación se volverá cada vez más lenta hasta congelarse o cerrarse inesperadamente.

3. Punto de Dolor Prioritario 3: [Dificultad para registrar el crecimiento del ganado — Cebon/Ternero/Novillo]

- **Trazabilidad a Hallazgo:** H-06, H-07, H-08 (secundariamente H-04)
- **Criterio de Priorización:** El sistema trata la etapa de vida del animal (ternero, novillo, cebón) de forma fija e inflexible. Cuando un ternero crece, la aplicación genera un error en lugar de actualizar su estado, lo que obliga a borrar el animal y crearlo de nuevo, perdiendo todo el historial de su peso, vacunas e inversión realizada.

4. Punto de Dolor Prioritario 4: Concentración Excesiva de Tareas en el Administrador Principal (Clase Hacienda)

- **Trazabilidad a Hallazgo:** H-09(secundariamente H-10, H-11, H-12)
- **Criterio de Priorización:** Esta clase se encarga de controlar prácticamente todo el negocio al mismo tiempo (ventas, vacunación, potreros). Esto hace que realizar modificaciones o agregar nuevas funciones sea un proceso demorado, costoso y propenso a generar fallas en zonas del sistema que ya estaban funcionando bien.

FASE 2 — LOS CAMBIOS QUE VIENEN (ANÁLISIS DE IMPACTO AS-IS)

2.1 Catálogo de Solicitudes de Cambio

El negocio tiene aprobadas y priorizadas las siguientes solicitudes para el próximo trimestre:

Código	Solicitudes de Cambio Aprobadas (Hacienda / Farmacia)
SC-1	La hacienda en el futuro va a comenzar a vender productos derivados del ganado como: lácteos, carne, piel.
SC-2	La hacienda tiene la necesidad de conectar a las reses chips para la geolocalización
SC-3	Además de las vacunas, se va a requerir tener la historia clínica de cada res en un futuro

2.2 Matriz de Impacto en Arquitectura Actual (AS-IS)

Para cada una de las solicitudes de cambio, se realiza el análisis sobre el código heredado:

Código SC	Solicitud Analizada	¿Cuántas Clases/Archivos se Modificarían Hoy?	Comportamiento Existente en Riesgo de Romperse
SC-1	La hacienda comenzará a vender productos derivados del ganado (lácteos, carne, piel), no solo la res completa	Hacienda.cs: Es donde vive <code>vender_res</code> y la lista <code>l_ventas</code> tendrías que agregar la lógica nueva de venta de derivados ahí mismo. Venta.cs: La clase actual asume que una venta es de una res completa; no	Hoy el sistema solo sabe vender la res completa y sacarla del potrero. Si se agrega la opción de vender derivados (leche, carne, piel) sin quitar la res, y ambos tipos de venta terminan guardándose juntos en la misma lista, los reportes o cálculos de ventas podrían mezclar cosas distintas (ventas que sí quitan la res con ventas que no) y salir mal

		tiene forma de representar "se vendieron 50 kg de carne" o "se vendió leche" 3. Res.cs: Necesitarías saber cuánto rinde cada res en lácteos/carne/piel ese dato no existe hoy en ningún atributo	
SC-2	Se requiere conectar chips a las reses para habilitar geolocalización	1. Res.cs: Es donde se agregaría los atributos nuevos 2. Ternero.cs: La clase hereda de res.cs , y ya sobrescribe la edad con su propia lógica, tocaría como revisar que agregar un atributo nuevo no dañe esa lógica 3. Cebon.cs: Sucede lo mismo que Ternero 4. Novillo.cs: Pasa lo mismo que ternero y cebón	Cada tipo de res (ternero, cebón, novillo) ya tiene su propia forma de validar sus datos y puede lanzar errores distintos. Si se agrega la ubicación del chip, corre el riesgo de heredar ese mismo problema y fallar de forma distinta según el tipo de res
SC-3	Además del historial de vacunas, se necesita registrar la historia clínica completa de cada res	1. Res.cs: Se necesitaría agregar una lista nueva, para que	El método que aplica vacunas ya hace varias cosas a la vez (cuenta vacunas, válida tipos, dispara avisos). Meter

		tenga el historial 2. Hacienda.cs : En esta clase tocaría agregar un método nuevo como AgregarHistorial	ahí la historia clínica nueva aumenta el riesgo de dañar sin querer el conteo de vacunas o los avisos que ya funcionan
--	--	---	---

FASE 3 — DISEÑO DE LA NUEVA ARQUITECTURA (TO-BE)

3.1 Diagrama UML de la Nueva Arquitectura (TO-BE)

El diagrama UML “Mejorado” se encuentra adjunto en el repositorio.

3.2 Justificación de la Aplicación de los 5 Principios SOLID

Single Responsibility Principle (SRP)

- ¿Qué clase original del sistema AS-IS se dividió en partes?

La principal clase que se identificó como incumplimiento del principio de Responsabilidad Simple (SRP) fue la clase Hacienda. En el diseño original la clase Hacienda concentraba múltiples responsabilidades del sistema:

 Hacienda
<pre>-l_potreros : List<Potrero> -l_ventas : List<Venta> -l_vacunas : List<Vacuna> -publisher_vacunacion_completa : PublisherVacunacionCompletada -publisher_vacuna_vendida : PublisherVacunaVendida -publisher_peso_min : PublisherPesoMin -publisher_peso_ideal : PublisherPesoVenta +Hacienda() +crear_potrero(identificacion: string, tipo_potrero: l_tipos_potreros) : string +buscar_potrero(nombre: string) : Potrero +anadir_res_potrero(id_potrero: string, nombre: string, edad: ushort, peso: uint) : string +vender_res(id_potrero: string, nombre: string, monto: uint) : string +alimentar_res(id_potrero: string, nombre: string) : string +alimentar_res(id_potrero: string, nombre: string, cantidadAlimento: uint) : string +crear_vacuna(nombre: string, lote: string, fecha_vencimiento: DateTime, fecha_aplicacion: DateTime, grado_atenuacion: enum_l_atenuaciones) : string +crear_vacuna(nombre: string, lote_base: string, fecha_vencimiento: DateTime, fecha_aplicacion: DateTime, periodo_aplicacion: uint, cantidad: uint) : string +crear_vacuna(nombre: string, lote_base: string, fecha_vencimiento: DateTime, fecha_aplicacion: DateTime, grado_atenuacion: enum_l_atenuaciones, cantidad: uint) : string +aplicar_vacuna(vacuna: Vacuna, nombre: string, id_potrero: string) : string ~EventHandler() : void</pre>

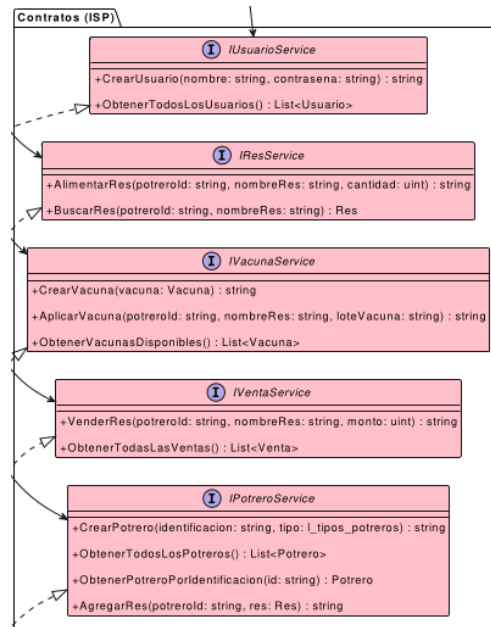
Esto significaba que Hacienda no solamente representaba la hacienda ganadera como entidad del dominio, sino que también ejecutaba directamente operaciones relacionadas con potreros, alimentación, ventas y vacunación. Por lo que generaba que tuviera muchas razones para cambiar.

- ¿En cuántas nuevas clases se segmenta la lógica?

La lógica que originalmente estaba centralizada en Hacienda se distribuyó principalmente en cuatro servicios especializados:

1. **PotreroService:** Creación de potreros, consulta de potreros y agregación de reses a un potrero.
2. **ResService:** Búsqueda y gestión de las reses. **VacunaService:** Creación de vacunas, aplicación de vacunas y consulta de vacunas disponibles.
3. **VentaService:** Venta de reses y consulta de las ventas realizadas.

En el diseño mejorado, estas operaciones se exponen mediante interfaces.



- **¿Qué beneficio tangible de negocio o mantenimiento se obtiene con este cambio?**

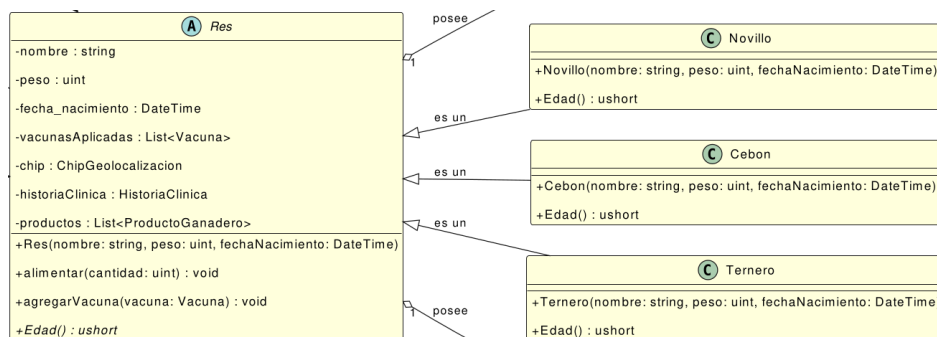
El principal beneficio es la reducción del impacto de los cambios. En el diseño AS-IS, una modificación en cualquiera de los procesos de la hacienda podía obligar a modificar la clase Hacienda, debido a que todos estos procesos estaban centralizados en ella. Esto aumentaba el riesgo de introducir errores en funcionalidades que no estaban relacionadas con el cambio.

Open/Closed Principle (OCP)

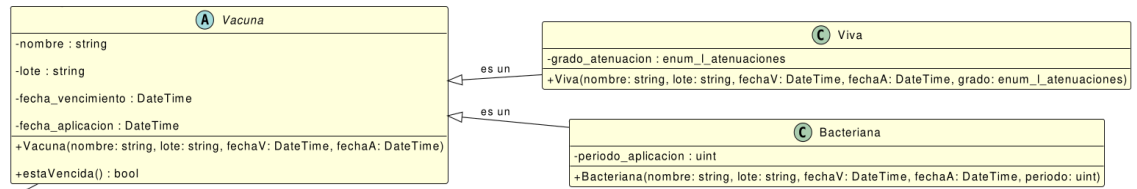
- **¿Qué partes del sistema se han hecho 'abiertas a la extensión pero cerradas a la modificación'?**

En el proyecto, las partes principales que dejamos abiertas a la extensión pero cerradas a la modificación (OCP) son las siguientes:

- La clase abstracta “Res” define la estructura básica, si en el futuro la hacienda decide agregar un nuevo tipo de res como un toro, etc, solo debemos de crear una nueva subclase sin tener que modificar la clase res ni cambiar la lógica en potrero o hacienda.



- La clase abstracta “Vacuna” maneja los datos generales. Si se quiere incluir una nueva categoría de vacuna, simplemente se extiende de vacuna si tocar el código existente



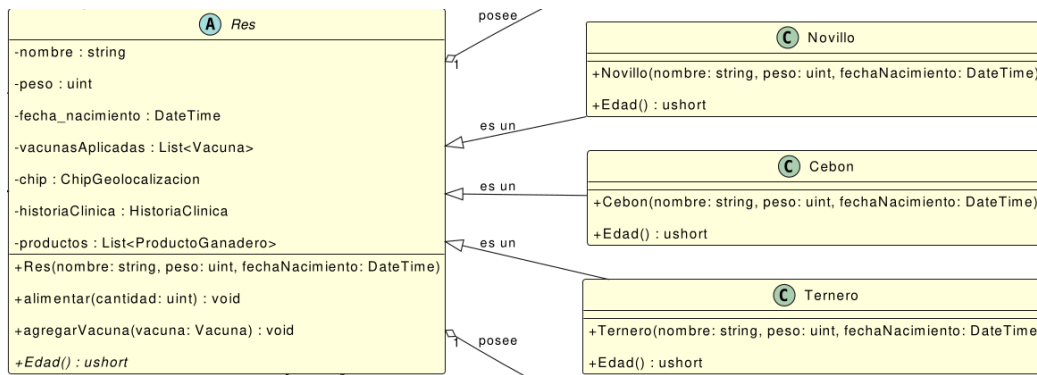
- Ya que se escogió la solución SC-1 para implementar el sistema de ventas se abstraee mediante la interfaz **IProductosVenta**, lo que permite comercializar nuevos productos sin modificar la clase venta

• ¿Cómo se asimilan las solicitudes de cambio futuras (SC-1, SC-2, etc.) sin reescribir código existente?

- SC-1: esta se resuelve creando nuevas clases como ProductosLacteo, ProductoCarne o ProductoPiel que implementen la interfaz IProductosVenta. Las clases venta, hacienda y VentaService siguen funcionando exactamente igual sin cambiar una sola línea del código
- SC-2: Se asimila añadiendo una interfaz IGeolocalizable o asociando un componente de localización a Res sin alterar los potreros existentes.
- SC-3: Se incluye creando puede ser una entidad o un servicio HistoriaClinica que se asocie a la res, sin modificar la forma en que se aplican las vacunas actualmente, en la que se puede llamar, en nombre de la res, la edad, una lista de las vacunas aplicadas, etc.

Liskov Substitution Principle (LSP)

- Justificación de la validez de cada jerarquía de herencia (nueva o conservada)
 - La jerarquía que está manejando res con ternero, novillo y cebón es totalmente válida porque cumple con la relación “es un tipo de”. Un ternero, un novillo y un cebón son reses que comparten atributos claves como (nombre, peso, edad, L_vacunas_aplicadas). Lo único que cambia en cada subclase es la regla que valida la edad máxima (Edad), sin alterar las firmas de los métodos ni la forma en que potero o hacienda interactúan con ellas.



- La jerarquía vacuna con bacteria y viva, es válida porque las vacunas bacterianas y vivas comparten atributos de una vacuna como nombre, lote, fecha_vencimiento, fecha_aplicacion. Aunque agregan datos específicos como periodo_aplicacion en bacteriana y enum_l_atenuaciones en viva, cualquier servicio o métodos como aplicar_vacuna(Vacuna v,...) las trata indistintamente sin necesidad de saber el tipo exacto

Tabla de Verificación de Sustituibilidad (LSP)

Clase Base / Subclase	Precondiciones (¿Se mantienen/debil itan?)	Postcondiciones (¿Se mantienen/refue rzan?)	Invariantes (¿Se preservan?)	Excepciones (¿Son compatibles?)
RES -> TERNERO	Sí: Se mantienen. Requiere un nombre no vacío, edad y peso válidos igual que Res.	Sí: Se refuerzan. Garantiza que la edad asignada no supere ReglaRes.edad_max_ternero.	Sí: Preserva la coherencia interna de la entidad Res y su lista de vacunas.	Sí: Lanza Exception por límite de edad, compatible con las validaciones de negocio.
RES -> NOVILLO	Sí: Se mantienen. Acepta los parámetros estándar de creación de una Res.	Sí: Se mantienen. Asigna la edad respetando el rango superior para la etapa de desarrollo.	Sí: Mantiene intactos el peso, nombre y la lista de vacunas aplicadas.	Sí: Mantiene el mismo tipo de excepción manejada por las validaciones de Res.
RES -> CEBON	Sí: Se mantienen. Requiere la misma estructura de datos de la superclase.	Sí: Se refuerzan. Verifica que la edad esté dentro del rango de ceba	Sí: Preserva la integridad del animal para el cálculo de consumo de alimento y peso.	Sí: Compatible con el flujo de excepciones del sistema.

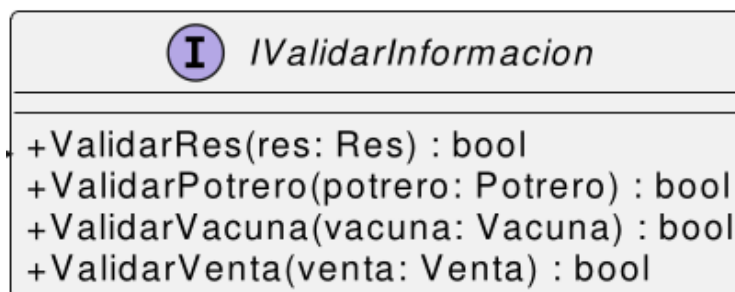
		(ReglaRes.edad_max_cebon).		
VACUNA -> BACTERIANA	Sí: Se mantienen. Requiere nombre, lote y fechas de vencimiento/aplicación válidas.	Sí: Se refuerzan. Valida adicionalmente que el período de aplicación esté en el rango legal de semanas.	Sí: Mantiene la vigencia y el lote como identificadores únicos de la vacuna.	Sí: Las excepciones por vencimiento o rango no rompen la colección de inventario.
VACINA -> VIVA	Sí: Se mantienen. Mantiene las mismas precondiciones de registro que la clase base Vacuna.	Sí: Se mantienen. Asigna el grado de atenuación según el enum predeterminado.	Sí: Preserva la integridad de la fecha de vencimiento y lote de la vacuna.	Sí: No genera excepciones que rompan el procesamiento en aplicar_vacuna.

Para asegurar el cumplimiento de LSP, evalúe la sustituibilidad de las subclases mediante este checklist:

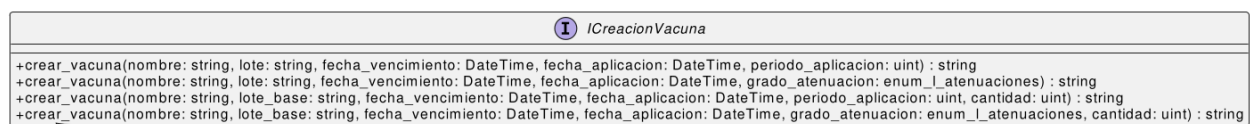
Interface Segregation Principle (ISP)

- ¿Qué interfaces se crearon para evitar que los clientes dependan de métodos que no utilizan?

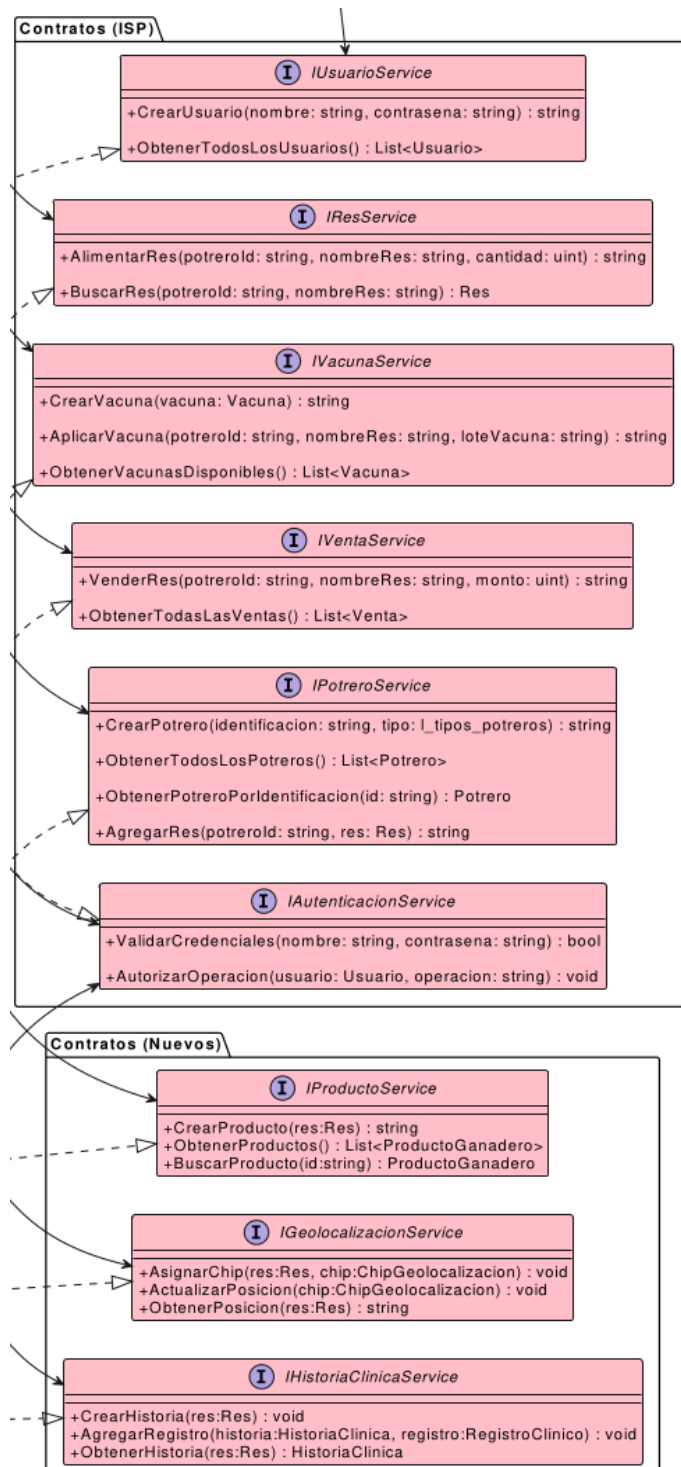
En el sistema original se identificó que algunas interfaces agrupaban operaciones de diferentes responsabilidades. Por ejemplo: IValidarInformación concentraba métodos para validar diferentes entidades:



De manera similar, ICreaciónVacuna agrupaba múltiples sobrecargas de crear_vacuna() para diferentes tipos de vacunas cantidades:



En el diseño mejorado se decidió segregar las interfaces de acuerdo con el servicio y la responsabilidad que representan. Entre las principales creadas o reorganizadas están:



Dependency Inversion Principle (DIP)

- En el sistema AS-IS, los servicios dependían directamente de una implementación concreta de persistencia. Por ejemplo, PotreroService tenía una dependencia directa hacia PersistenciaService. Esto generaba un acoplamiento entre la lógica de negocio y la forma específica en que se almacenaban los datos. En el diseño TO-BE, se invierte esta dependencia mediante interfaces. El flujo queda de la siguiente manera: **Módulo de alto nivel** → **Interfaz/abstracción** ← **Módulo de bajo nivel**

Por ejemplo:

Módulo de Alto Nivel	Módulo de Bajo Nivel	Abstracción (Interfaz/Base)
UsuarioService	PersistenciaTxtService	IPersistenciaUsuarios
PotreroService	PersistenciaTxtService	IPersistenciaHacienda

3.3 Registros de Decisión Arquitectónica (ADR)

A continuación se detallan las decisiones estructurales más relevantes del proyecto:

ADR-01: [so de Herencia para los nuevos productos (SC-H1)]

Atributo de Decisión	Detalle y Argumentación Técnica
Contexto y Evidencia	Para cumplir con el requerimiento SC-H1 (vender lácteos, carne, piel), vimos que el código original solo estaba pensado para vender reses completas (usando IVentaRes). Si dejábamos el código así, íbamos a tener que modificar la clase principal de ventas cada vez que la hacienda sacara un producto nuevo, rompiendo el principio Open/Closed (OCP).
Alternativas Evaluadas	Alternativa A (Seleccionada): Crear una clase base o interfaz llamada ProductoGanadero. De ahí hacemos que hereden Carne, piel y Lacteo (que a su vez deriva en Leche, Queso y se le pueden agregar más productos a futuro). Así, el módulo de ventas solo interactúa con el concepto general de "Producto" sin importarle qué tipo exacto es. Alternativa B (Descartada): Pensamos en usar el

	Patrón Estrategia (Strategy). Es decir, tener una sola clase genérica Producto y pasarle por inyección una interfaz de comportamiento (ej: IComportamientoVenta con implementaciones como VentaCarne, VentaLeche). Esta opción es excelente y cumple OCP perfecto, ya que si nos piden vender "Lana", solo creamos una clase VentaLana sin tocar la base.
Decisión Tomada y Justificación	Elegimos la A ya que con la B los productos ganaderos tienen datos muy distintos (la carne tiene "tipo de corte", la leche tiene "temperatura de refrigeración"). Si usáramos Strategy, la clase única Producto tendría que tener todos esos atributos mezclados, o usar un diccionario genérico muy enredado. La herencia (Alternativa A) permite que cada producto tenga solo los atributos que de verdad le corresponden.
Principios SOLID Involucrados	OCP, LSP
Consecuencias / Costos Aceptados	Al usar polimorfismo, el servicio de ventas debe estar muy bien diseñado para aceptar cualquier ProductoGanadero sin tener que andar verificando el tipo exacto

ADR-02: [Reducción de responsabilidades de la clase Hacienda]

Atributo de Decisión	Detalle y Argumentación Técnica
Contexto y Evidencia	En el código heredado, la clase Hacienda procesaba prácticamente toda la lógica de negocio. Implementaba múltiples interfaces inmensas (vacunación, ventas, reportes). Necesitábamos separar esto para no romper el sistema con cada requerimiento nuevo.
Alternativas Evaluadas	Alternativa A (Seleccionada): Mover la lógica de negocio a servicios de aplicación específicos (VacunaService, VentaService, ProductoService) y a clases de reglas de dominio (ReglaVacuna, ReglaRes, ReglaPotrero). La clase Hacienda queda únicamente como una entidad de dominio que representa los datos de la finca.

	Alternativa B (Descartada): Implementar Eventos de Dominio (Domain Events). En lugar de usar servicios directos, hacíamos que cualquier acción disparara un evento (ej: AplicarVacunaCommand lanza VacunaAplicadaEvent) procesado por manejadores independientes.
Decisión Tomada y Justificación	Aunque la opción B cumple SOLID, los sistemas por eventos fragmentan la trazabilidad. Para nuestro equipo, el flujo explícito de los Servicios de Aplicación y las Reglas de Dominio (Alternativa A) es mucho más seguro y fácil de depurar.
Principios SOLID Involucrados	SRP, OCP, ISP, DIP
Consecuencias / Costos Aceptados	Obligó a ajustar los controladores (VacunaController, VentaController, ProductoController) para que dependan de sus servicios correspondientes (IVacunaService, IVentaService, IProductoService) en lugar de llamar a Hacienda.

ADR-03: [Inversión de dependencias en la persistencia de Productos]

Atributo de Decisión	Detalle y Argumentación Técnica
Contexto y Evidencia	Para manejar el almacenamiento de los nuevos productos derivados (SC-H1), los servicios de aplicación necesitaban interactuar con la infraestructura. Depender de la clase concreta de persistencia violaba el principio DIP.
Alternativas Evaluadas	Alternativa A (Seleccionada): Crear un puerto o interfaz dedicada en la capa de puertos llamada IPersistenciaProductos, la cual es implementada por la clase de infraestructura PersistenciaTxtService. Alternativa B (Descartada): Implementar un Repositorio Genérico (IRepository<T>) único para todas las entidades del sistema.
Decisión Tomada y Justificación	Descartamos la B porque impone un contrato único CRUD que no nos permite definir métodos

	de consulta específicos para productos ganaderos sin obligar a las demás entidades a tener métodos que no necesitan (violando ISP).
Principios SOLID Involucrados	DIP, ISP
Consecuencias / Costos Aceptados	Requiere registrar explícitamente la interfaz IPersistenciaProductos con su implementación en el contenedor de inyección de dependencias

ADR-04: [Refactorización y aislamiento de reglas en la clase Potrero]

Atributo de Decisión	Detalle y Argumentación Técnica
Contexto y Evidencia	En el sistema original, la clase Potrero concentraba demasiadas responsabilidades: almacenaba su estructura de datos, manejaba la lógica de agregar reses, calculaba sus límites y disparaba eventos. Esta mezcla violaba directamente el Principio de Responsabilidad Única (SRP).
Alternativas Evaluadas	Alternativa A (Seleccionada): Separar las responsabilidades delegando la acción de agregar/mover directamente en la lógica de Res, y aislar la validación de capacidad creando la clase ReglaPotrero en la capa de reglas de dominio. De esta forma, ReglaPotrero solo se encarga de validar si hay capacidad disponible antes de que el animal sea agregado. Alternativa B (Descartada): Implementar el Patrón Estado (State Pattern), creando clases para representar el estado del potrero (PotreroVacioState, PotreroLlenoState) y delegar en ellas si se puede o no recibir un animal.
Decisión Tomada y Justificación	Descartamos la B porque la capacidad del potrero se resuelve con una validación matemática sencilla sobre un conteo límite. Introducir clases de estado agregaba una sobre-ingeniería innecesaria, mientras que aislar la validación en ReglaPotrero resuelve el problema de forma mucho más directa y clara.
Principios SOLID Involucrados	RP, OCP

Consecuencias / Costos Aceptados	Requiere una orquestación clara en el código para asegurar que siempre se invoque a ReglaPotrero para validar el límite antes de ejecutar el cambio de potrero en el objeto Res.
---	--

ADR-05: [Herencia en Potrero]

Atributo de Decisión	Detalle y Argumentación Técnica
Contexto y Evidencia	En el sistema original, un animal pasa por distintas etapas (Ternero, Novillo, Cebón), cada una con reglas de negocio diferentes (ej: un ternero no se puede vender igual que un cebón, y tienen pesos mínimos distintos). Dejándolos en una sola clase Res, estaba llena de condicionales (if (edad < X) o if (tipo == "Ternero")) para saber qué reglas aplicarle, violando OCP y SRP.
Alternativas Evaluadas	Alternativa A (Seleccionada): Usar Herencia (Polimorfismo). Mantener a Res como clase base y crear subclases específicas: Ternero, Novillo y Cebon. Cada subclase encapsula sus propios rangos de edad, peso y reglas específicas. Alternativa B (Descartada): Implementar el Patrón Estado (State Pattern). Consistía en tener una única clase concreta Res, y pasarle por inyección una interfaz IEstadoDesarrollo (con implementaciones como EstadoTernero, EstadoNovillo).
Decisión Tomada y Justificación	Descartamos la B porque el patrón Estado nos obligaba a programar un "motor de transiciones" complejo (lógica para mutar el estado automáticamente cuando el animal cumple años o sube de peso). Para el alcance actual de modernización de la Hacienda, la herencia (Alternativa A) es mucho más directa de implementar y consultar.
Principios SOLID Involucrados	SRP, OCP
Consecuencias / Costos Aceptados	Al usar herencia para etapas de vida, si un animal crece y pasa de Novillo a Cebon, técnicamente en código implica tener que instanciar un nuevo objeto Cebon copiando los

	datos del Novillo anterior, en lugar de solo cambiarle un atributo. Aceptamos este costo porque priorizamos la limpieza de las reglas de validación en cada clase.
--	--

FASE 4 — IMPLEMENTACIÓN Y EVIDENCIA

4.1 Código Fuente y Repositorio

Enlace al Repositorio de GitHub del Proyecto: (Todo los recursos y el código se encuentran en la rama *development*)

https://github.com/Diego018/proyecto_hacienda

Instrucciones Rápidas de Compilación y Ejecución:

1. Clonar el repositorio:

Comando: git clone https://github.com/Diego018/proyecto_hacienda

```
cd proyecto_hacienda
```

2. Restaurar dependencias y compilar toda la solución:

Comando: dotnet build proyecto_hacienda.slnx

3. Ejecutar el proyecto principal (p_mvchacienda), desde su propia carpeta:

Comandos: cd p_mvchacienda

```
dotnet run
```

4.2 Evidencia de Preservación del Comportamiento

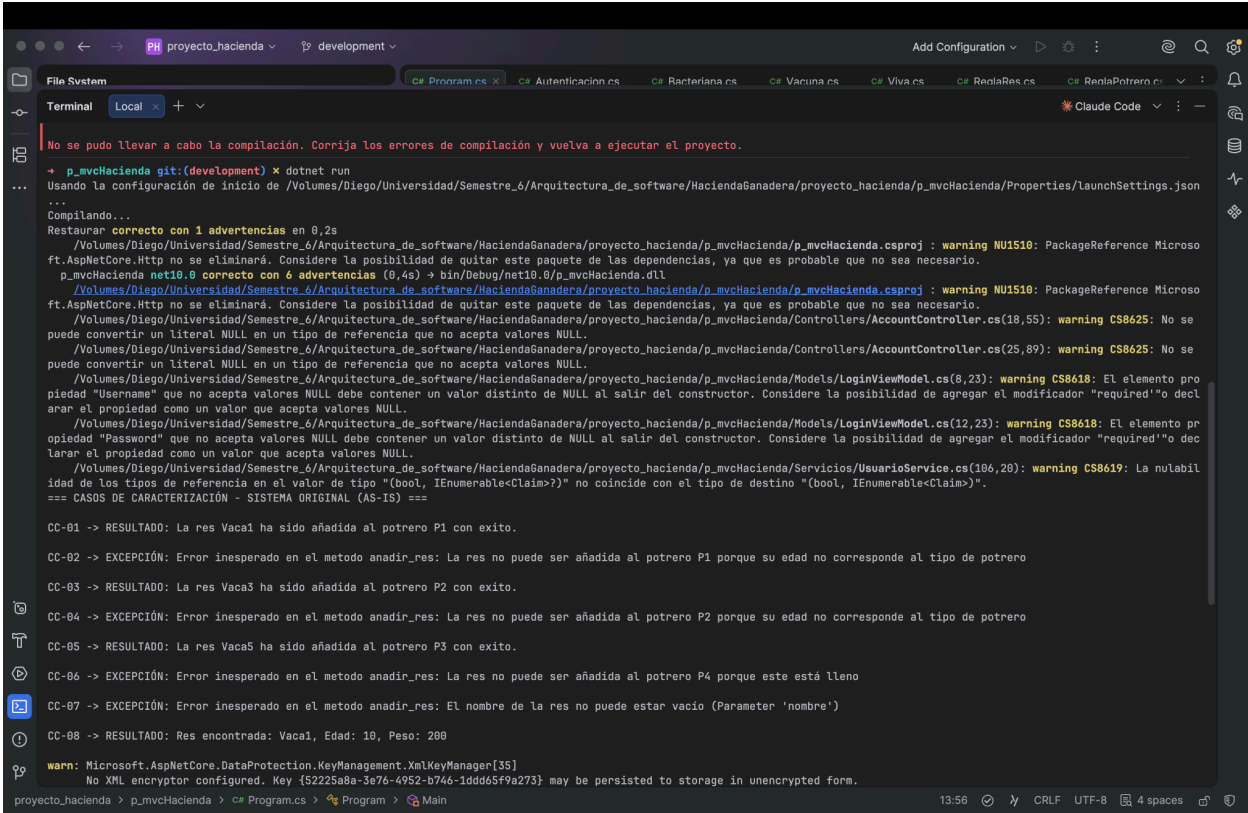
Los siguientes 8 casos de caracterización demuestran empíricamente que el comportamiento observable no sufrió variaciones entre la versión original (AS-IS) y la rediseñada (TO-BE):

ID	Escenario de Prueba	Parámetros Entrada	Salida Original (AS-IS)	Salida Rediseñado (TO-BE)	¿Coinciden ? (S/N)
CC-01	Añadir ternero con edad válida	nombre="Vaca1", edad=10, peso=200, potrero tipo ternero	La res Vaca1 ha sido añadida al potrero P1 con éxito.	La res 'Vaca1' fue añadida al potrero 'P1'.	Sí

CC-02	Añadir ternero con edad fuera de rango	nombre="Vaca2", edad=20, peso=200, potrero tipo ternero	Excepción: Error inesperado en el metodo anadir_res: La res no puede ser añadida al potrero P1 porque su edad no corresponde al tipo de potrero	EXCEPCIÓN: La res 'Vaca2' no cumple las condiciones de peso/edad para su categoría.	Sí
CC-03	Añadir cebón con edad válida	nombre="Vaca3", edad=30, peso=350, potrero tipo cebón	La res Vaca3 ha sido añadida al potrero P2 con éxito.	La res 'Vaca3' fue añadida al potrero 'P2'.	Sí
CC-04	Añadir cebón con edad menor al mínimo	nombre="Vaca4", edad=5, peso=350, potrero tipo cebón	Excepción: Error inesperado en el metodo anadir_res: La res no puede ser añadida al potrero P2 porque su edad no corresponde al tipo de potrero	EXCEPCIÓN: La res 'Vaca4' no cumple las condiciones de peso/edad para su categoría.	Sí
CC-05	Añadir novillo con edad válida	nombre="Vaca5", edad=55, peso=450, potrero tipo novillo	La res Vaca5 ha sido añadida al potrero P3 con éxito.	La res 'Vaca5' fue añadida al potrero 'P3'.	Sí
CC-06	Añadir res a potrero ya lleno	Potrero con 150 reses, intentar una más	Excepción: Error inesperado en el metodo anadir_res: La res no puede ser añadida al potrero P4 porque este está lleno	EXCEPCIÓN: El potrero 'P4' alcanzó su capacidad máxima (150 reses).	Sí

CC-07	Añadir res con nombre vacío	nombre="", edad=10, peso=200	peso=200Excepción: Error inesperado en el metodo anadir_res: El nombre de la res no puede estar vacío (Parameter 'nombre')	EXCEPCIÓN: El nombre de la res no puede estar vacío.	Sí
CC-08	Buscar una res existente por nombre	buscar_res("Vaca1")	Res encontrada: Vaca1, Edad: 10, Peso: 200	Res encontrada: Vaca1, Edad: 10, Peso: 200	Sí

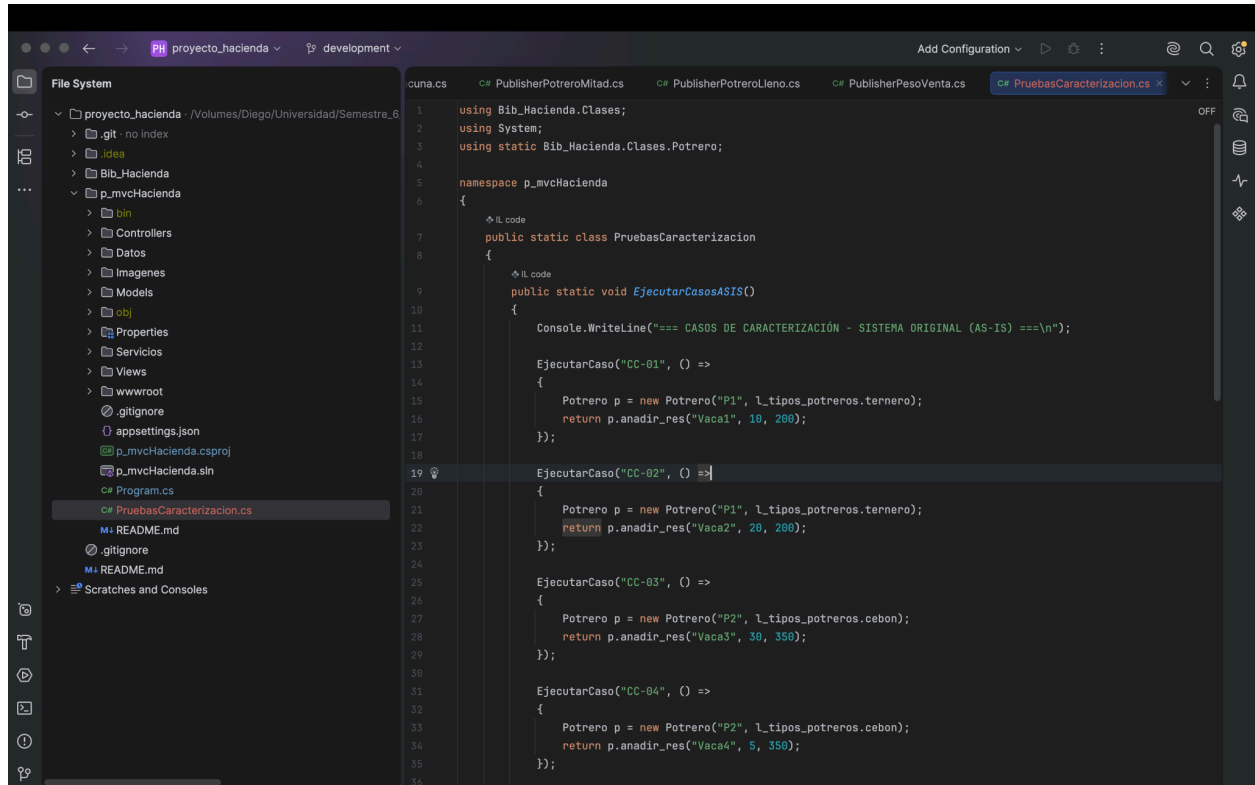
FOTO DE EVIDENCIA



Se creó un archivo de pruebas (Pruebas Caracterización.cs) dentro del proyecto principal, con un método que ejecuta automáticamente los 8 escenarios definidos: crea potreros de cada tipo, intenta añadir reses con edades válidas e inválidas, prueba el límite de capacidad del potrero, valida el manejo de un nombre vacío, y realiza una búsqueda de una res ya existente. Cada caso se ejecuta de forma controlada,

capturando tanto el resultado exitoso como cualquier excepción que se produzca, y se imprime en consola de forma clara para poder documentar la salida real del sistema.

Este archivo se ejecuta al iniciar la aplicación, mostrando en la terminal el resultado de cada uno de los 8 casos antes de que el sistema continúe con su funcionamiento normal, esto permitió capturar evidencia real y verificable del comportamiento del sistema, en lugar de describirlo de forma teórica.



The screenshot shows the Visual Studio Code interface. On the left, the 'File System' pane displays the project structure for 'proyecto_hacienda'. The main editor area shows the code for 'PruebasCaracterizacion.cs'. The code is in C# and defines a static class 'PruebasCaracterizacion' with a method 'EjecutarCasosASIS()' that contains four cases (CC-01 to CC-04) for testing the system's behavior.

```
1 using Bib_Hacienda.Clases;
2 using System;
3 using static Bib_Hacienda.Clases.Potrero;
4
5 namespace p_mvcHacienda
6 {
7     public static class PruebasCaracterizacion
8     {
9         public static void EjecutarCasosASIS()
10        {
11            Console.WriteLine("=== CASOS DE CARACTERIZACIÓN - SISTEMA ORIGINAL (AS-IS) ===\n");
12
13            EjecutarCaso("CC-01", () =>
14            {
15                Potrero p = new Potrero("P1", l_tipos_potreros.ternero);
16                return p.anadir_res("Vacac1", 10, 200);
17            });
18
19            EjecutarCaso("CC-02", () =>
20            {
21                Potrero p = new Potrero("P1", l_tipos_potreros.ternero);
22                return p.anadir_res("Vacac2", 20, 200);
23            });
24
25            EjecutarCaso("CC-03", () =>
26            {
27                Potrero p = new Potrero("P2", l_tipos_potreros.cebon);
28                return p.anadir_res("Vacac3", 30, 350);
29            });
30
31            EjecutarCaso("CC-04", () =>
32            {
33                Potrero p = new Potrero("P2", l_tipos_potreros.cebon);
34                return p.anadir_res("Vacac4", 5, 350);
35            });
36        }
37    }
38 }
```

```
7 public static class PruebasCaracterizacion
9     public static void EjecutarCasosASIS()
10
11     {
12         EjecutarCaso("CC-05", () =>
13         {
14             Potrero p = new Potrero("P3", 1_tipos_potreros.novillo);
15             return p.anadir_res("Vacaa5", 55, 450);
16         });
17
18         EjecutarCaso("CC-06", () =>
19         {
20             Potrero p = new Potrero("P4", 1_tipos_potreros.novillo);
21             for (int i = 0; i < 150; i++)
22             {
23                 p.anadir_res($"Res{i}", 55, 450);
24             }
25             return p.anadir_res("VacaaExtra", 55, 450);
26         });
27
28         EjecutarCaso("CC-07", () =>
29         {
30             Potrero p = new Potrero("P1", 1_tipos_potreros.terrero);
31             return p.anadir_res("", 10, 200);
32         });
33
34         EjecutarCaso("CC-08", () =>
35         {
36             Potrero p = new Potrero("P1", 1_tipos_potreros.terrero);
37             p.anadir_res("Vacaa1", 10, 200);
38             Res encontrada = p.buscar_res("Vacaa1");
39             return $"Res encontrada: {encontrada.Nombre}, Edad: {encontrada.Edad}, Peso: {encontrada.Peso}";
40         });
41     }
42
43     static void EjecutarCaso(string id, Func<string> accion)
44     {
45         try
46         {
47         }
48     }
49 }
```

```
7 public static class PruebasCaracterizacion
9     public static void EjecutarCasosASIS()
10
11     {
12         EjecutarCaso("CC-07", () =>
13         {
14             Potrero p = new Potrero("P1", 1_tipos_potreros.terrero);
15             return p.anadir_res("", 10, 200);
16         });
17
18         EjecutarCaso("CC-08", () =>
19         {
20             Potrero p = new Potrero("P1", 1_tipos_potreros.terrero);
21             p.anadir_res("Vacaa1", 10, 200);
22             Res encontrada = p.buscar_res("Vacaa1");
23             return $"Res encontrada: {encontrada.Nombre}, Edad: {encontrada.Edad}, Peso: {encontrada.Peso}";
24         });
25     }
26
27     static void EjecutarCaso(string id, Func<string> accion)
28     {
29         try
30         {
31             string resultado = accion();
32             Console.WriteLine($"{id} -> RESULTADO: {resultado}\n");
33         }
34         catch (Exception ex)
35         {
36             Console.WriteLine($"{id} -> EXCEPCIÓN: {ex.Message}\n");
37         }
38     }
39 }
```

Cabe resaltar que estas son las pruebas para el código viejo.

Pruebas de caracterización para el código refactorizado.

```
terminal dotnet + -
...
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvcHacienda/Models/LoginViewModel.cs(12,23): warning CS866
propiedad "Password" que no acepta valores NULL debe contener un valor distinto de NULL al salir del constructor. Considere la posibilidad de agregar el modificado
larar el propiedad como un valor que acepta valores NULL.
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvcHacienda/Controllers/VentaController.cs(41,36): warnin
convertir un literal nulo o un posible valor nulo en un tipo que no acepta valores NULL
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvcHacienda/Controllers/VentaController.cs(42,36): warnin
convertir un literal nulo o un posible valor nulo en un tipo que no acepta valores NULL
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvcHacienda/Controllers/VentaController.cs(43,41): warnin
argumento de referencia nulo para el parámetro "s" en "uint uint.Parse(string s)".
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvcHacienda/Controllers/VentaController.cs(45,41): warnin
argumento de referencia nulo para el parámetro "potreroId" en "string IVentaService.VenderRes(string potreroId, string nombreRes, uint monto)".
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvcHacienda/Controllers/VentaController.cs(45,52): warnin
argumento de referencia nulo para el parámetro "nombreRes" en "string IVentaService.VenderRes(string potreroId, string nombreRes, uint monto)".
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvcHacienda/Servicios/ResService.cs(72,20): warning CS860
valor devuelto de referencia nulo.
=== CASOS DE CARACTERIZACIÓN - SISTEMA REFACTORIZADO (TO-BE) ===

CC-01 -> RESULTADO: La res 'Vaca1' fue añadida al potrero 'P1'.
CC-02 -> EXCEPCIÓN: La res 'Vaca2' no cumple las condiciones de peso/edad para su categoría.
CC-03 -> RESULTADO: La res 'Vaca3' fue añadida al potrero 'P2'.
CC-04 -> EXCEPCIÓN: La res 'Vaca4' no cumple las condiciones de peso/edad para su categoría.
CC-05 -> RESULTADO: La res 'Vaca5' fue añadida al potrero 'P3'.
CC-06 -> EXCEPCIÓN: El potrero 'P4' alcanzó su capacidad máxima (150 reses).
CC-07 -> EXCEPCIÓN: El nombre de la res no puede estar vacío.
CC-08 -> RESULTADO: Res encontrada: Vaca1, Edad: 10, Peso: 200

Info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5231
Info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
Info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
Info: Microsoft.Hosting.Lifetime[0]
Content root path: /Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvcHacienda
```

Se actualizó el archivo de pruebas de caracterización para ejecutar 8 escenarios documentados en la versión original del sistema. Ahora invoca las clases y firmas de la nueva arquitectura, enfocándose en la construcción de res según su categoría usando fecha de nacimiento y validaciones delegadas a subclases sin condicionales.

Cada caso se probó, registrando el resultado y cualquier excepción en consola. Esto permitió comparar los resultados del sistema refactorizado con los de la versión original. Los resultados coincidieron en todos los casos: las reses válidas se añadieron correctamente, las inválidas fueron rechazadas, se respetó el límite de reses en el potrero, se mantuvo el rechazo de nombres vacíos y la búsqueda de res registrada devolvió los mismos datos.

```
projecto_hacienda - proyecto_hacienda.slnx
projecto_hacienda [Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto

C# IVacunaService.cs C# VentaService.cs C# UsuarioService.cs C# AutenticacionService.cs C# PruebasCaracterizacion.cs x C# ReglaRes.cs C# Cebon.cs C# Novillo.cs C# Regl: v ...

2 using Bib_Hacienda.Clases;
3 using Bib_Hacienda.Reglas;
4
5 namespace p_mvcHacienda {
6
7     public static class PruebasCaracterizacion {
8
9         public static void EjecutarCasosASIS() {
10
11             Console.WriteLine("=== CASOS DE CARACTERIZACIÓN - SISTEMA REFACTORIZADO (T0-BE) ===\n");
12
13             EjecutarCaso(id: "CC-01", accion: () => {
14                 Potrero p = new Potrero(id: "P1", tipo: l_tipos_potreros.Ternero);
15                 return AgregarConValidacion(p, new Ternero(nombre: "Vaca1", peso: 200, fechaNacimiento: DateTime.Now.AddMonths(-10)));
16             });
17
18             EjecutarCaso(id: "CC-02", accion: () => {
19                 Potrero p = new Potrero(id: "P1", tipo: l_tipos_potreros.Ternero);
20                 return AgregarConValidacion(p, new Ternero(nombre: "Vaca2", peso: 200, fechaNacimiento: DateTime.Now.AddMonths(-20)));
21             });
22
23             EjecutarCaso(id: "CC-03", accion: () => {
24                 Potrero p = new Potrero(id: "P2", tipo: l_tipos_potreros.Cebon);
25                 return AgregarConValidacion(p, new Cebon(nombre: "Vaca3", peso: 350, fechaNacimiento: DateTime.Now.AddMonths(-30)));
26             });
27
28             EjecutarCaso(id: "CC-04", accion: () => {
29                 Potrero p = new Potrero(id: "P2", tipo: l_tipos_potreros.Cebon);
30                 return AgregarConValidacion(p, new Cebon(nombre: "Vaca4", peso: 350, fechaNacimiento: DateTime.Now.AddMonths(-5)));
31             });
32
33             EjecutarCaso(id: "CC-05", accion: () => {
34                 Potrero p = new Potrero(id: "P3", tipo: l_tipos_potreros.Novillo);
35                 return AgregarConValidacion(p, new Novillo(nombre: "Vaca5", peso: 450, fechaNacimiento: DateTime.Now.AddMonths(-55)));
36             });
37         }
38     }
39 }

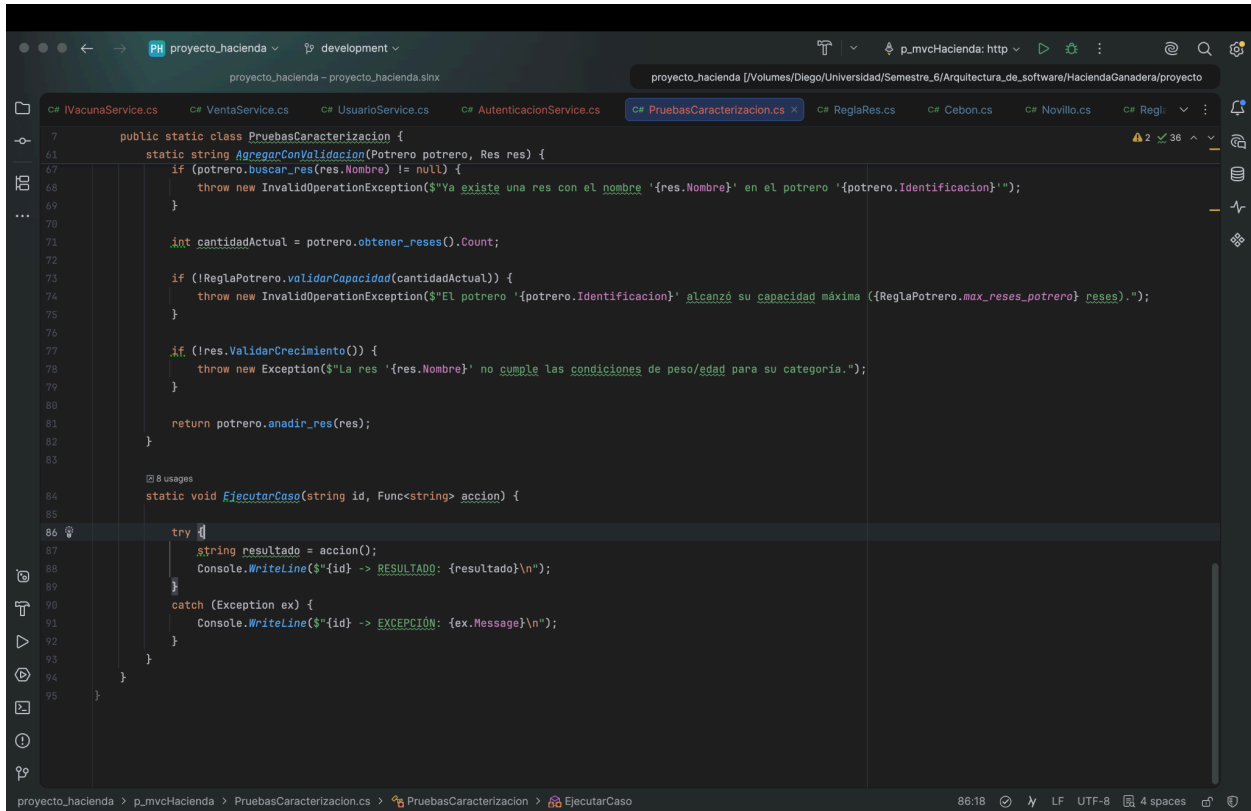
projecto_hacienda > p_mvcHacienda > PruebasCaracterizacion.cs > PruebasCaracterizacion > EjecutarCaso 86:18 y LF UTF-8 4 spaces
```

```
projecto_hacienda - proyecto_hacienda.slnx
projecto_hacienda [Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto

C# IVacunaService.cs C# VentaService.cs C# UsuarioService.cs C# AutenticacionService.cs C# PruebasCaracterizacion.cs x C# ReglaRes.cs C# Cebon.cs C# Novillo.cs C# Regl: v ...

7     public static class PruebasCaracterizacion {
8     public static void EjecutarCasosASIS() {
9
10         EjecutarCaso(id: "CC-06", accion: () => {
11             Potrero p = new Potrero(id: "P4", tipo: l_tipos_potreros.Novillo);
12             for (int i = 0; i < 150; i++) {
13                 AgregarConValidacion(p, new Novillo(nombre: $"Res{i}", peso: 450, fechaNacimiento: DateTime.Now.AddMonths(-55)));
14             }
15             return AgregarConValidacion(p, new Novillo(nombre: "VacaExtra", peso: 450, fechaNacimiento: DateTime.Now.AddMonths(-55)));
16         });
17
18         EjecutarCaso(id: "CC-07", accion: () => {
19             Potrero p = new Potrero(id: "P1", tipo: l_tipos_potreros.Ternero);
20             return AgregarConValidacion(p, new Ternero(nombre: "", peso: 200, fechaNacimiento: DateTime.Now.AddMonths(-10)));
21         });
22
23         EjecutarCaso(id: "CC-08", accion: () => {
24             Potrero p = new Potrero(id: "P1", tipo: l_tipos_potreros.Ternero);
25             AgregarConValidacion(p, new Ternero(nombre: "Vaca1", peso: 200, fechaNacimiento: DateTime.Now.AddMonths(-10)));
26             Res encontrada = p.buscar_res(nombre: "Vaca1");
27             return $"Res encontrada: {encontrada.Nombre}, Edad: {encontrada.Edad()}, Peso: {encontrada.Peso}";
28         });
29     }
30
31     // Replica las validaciones que aplica PotreroService.AgregarRes,
32     // ya que Potrero.anadir_res por sí solo no valida nada (SRP).
33     static string AgregarConValidacion(Potrero potrero, Res res) {
34         if (res == null || string.IsNullOrWhiteSpace(res.Nombre)) {
35             throw new ArgumentException("El nombre de la res no puede estar vacío.");
36         }
37
38         if (potrero.buscar_res(res.Nombre) != null) {
39             throw new InvalidOperationException($"Ya existe una res con el nombre '{res.Nombre}' en el potrero '{potrero.Identificacion}'");
40         }
41     }
42 }

projecto_hacienda > p_mvcHacienda > PruebasCaracterizacion.cs > PruebasCaracterizacion > EjecutarCaso 86:18 y LF UTF-8 4 spaces
```



```
7 public static class PruebasCaracterizacion {
61 static string AgregarConValidacion(Potrero potrero, Res res) {
67     if (potrero.buscar_res(res.Nombre) != null) {
68         throw new InvalidOperationException($"Ya existe una res con el nombre '{res.Nombre}' en el potrero '{potrero.Identificacion}'");
69     }
70
71     int cantidadActual = potrero.obtener_reses().Count;
72
73     if (!ReglaPotrero.validarCapacidad(cantidadActual)) {
74         throw new InvalidOperationException($"El potrero '{potrero.Identificacion}' alcanzó su capacidad máxima ({ReglaPotrero.max_reses_potrero} reses).");
75     }
76
77     if (!res.ValidarCrecimiento()) {
78         throw new Exception($"La res '{res.Nombre}' no cumple las condiciones de peso/edad para su categoría.");
79     }
80
81     return potrero.anadir_res(res);
82 }
83
84 static void EjecutarCaso(string id, Func<string> accion) {
85
86     try {
87         string resultado = accion();
88         Console.WriteLine($"{id} -> RESULTADO: {resultado}\n");
89     }
90     catch (Exception ex) {
91         Console.WriteLine($"{id} -> EXCEPCIÓN: {ex.Message}\n");
92     }
93 }
94 }
95 }
```

4.3 Prueba de Extensibilidad y Métricas Comparativas

Justificación de la Solicitud de Cambio Implementada:

El equipo seleccionó la solicitud SC-1 "La hacienda en el futuro va a comenzar a vender productos derivados del ganado como: lácteos, carne, piel."

Se escogió la SC-1 ya que representa una evolución muy real del negocio, pasar de vender únicamente ganado vivo a comercializar otros productos como leche, carne o pieles.

Desde el punto de vista de la arquitectura de software, esta solicitud es la más indicada para probar lo que es la extensibilidad y la evolución en la aplicación. En el código original, el sistema de ventas estaba amarrado solo a las reses. Al nosotros escoger la SC-1, pudimos comprobar si nuestra arquitectura propuesta para rediseñar el código realmente cumple con el principio open/closed, que nos permitirá agregar nuevos tipos de productos simplemente creando clases nuevas, sin tener que modificar o dañar el código que ya funciona y permitiendo la extensibilidad en el proceso

Métrica Comparativa de Esfuerzo e Impacto:

Arquitectura	Clases Creadas (Nuevas)	Clases Modificadas (Existentes)	Total Archivos Intervenidos	Costo o Tiempo de Implementación
Original (AS-IS)	0 (No extensible)	6	6	Alto: tocaría modificar directamente varias clases del código base como venta, hacienda, lVentaRes, VentaService, VentaController, ValidarVenta, cambiar los parámetros y hacer bastantes pruebas para no dañar la lógica existente en el código
Rediseñada (TO-BE)	4	0	4	Bajo: se implementa fácilmente creando una interfaz “nombre de la interfaz creada para los productos” y agregando las clases d los nuevos productos como, ProductosLacteo, ProductoCarne, ProductoPiel, sin tocar el código actual

4.4 Bitácora de Uso de Inteligencia Artificial (IA)

Registro estructurado de consultas y decisiones tomadas en co-creación con las herramientas de IA:

ID	Qué propuso la IA	Qué decidió el equipo (Aceptar / Refutar / Corregir)	Argumento Técnico del Equipo / Justificación
B-01	Sugerencia de aplicar herencia en todas las entidades de servicios.	RECHAZADO	Se prefirió usar composición para evitar una jerarquía rígida que violara LSP al heredar métodos no requeridos.
B-02	Se pidió ayuda en lo que es la graficación de los diagramas UML en código para exportarlos a una página web o a una plataforma que nos facilitara la diagramación de estos	Aceptado	Se aceptó el código que la IA nos proporcionó en los dos casos de los diagramas UML (el actual y el mejorado) y después se hizo una revisión exhaustiva de la corrección de relaciones, herencias, etc.
B-03	Se hizo una consulta para la pregunta del índice 3.2 “¿Cómo se asimilan las solicitudes de cambio futuras (SC-1, SC-2, etc.) sin reescribir código existente?” ya que se tenía una duda de como se podría implementar la geolocalización con chips	Aceptado	La solución que nos proporcionó la IA tiene lógica a como se tiene pensando reconstruir la arquitectura del proyecto
B-04	Se hizo una consulta para reducir la información que se tenía de la justificación de liskov para organizarla y adjuntarla en la tabla de verificación de sustituibilidad	Aceptado	Se aceptó una opción que nos dio la IA de la reducción de datos ya que al analizarlo no cambia ninguna información que le proporcionamos

B-05	Crear interfaces independientes para operaciones como creación de vacunas, vacunación y venta de reses, en lugar de concentrarse en una única interfaz.	ACEPTADO	La separación permite que cada cliente dependa únicamente de las operaciones que necesita. Esto mejora la cohesión de las interfaces y cumple con Interface Segregation Principle (ISP).
B-06	Utilizar una jerarquía de herencia para representar los productos de la hacienda, colocando a Producto como clase principal y a Lácteos como nivel intermedio para productos como leche y queso.	ACEPTADO	Se consideró que la jerarquía debía representar correctamente las relaciones del dominio. Se mantuvo la posibilidad de reutilización mediante herencia, pero se revisaron las responsabilidades de cada nivel para evitar que las subclases heredaran comportamientos que no les correspondieran, manteniendo LSP.
B-07	Revisar el diagrama completo para identificar violaciones de los principios SOLID y proponer modificaciones estructurales.	ACEPTADO	El equipo utilizó la revisión como apoyo para detectar problemas de SRP, ISP y LSP. Las modificaciones fueron evaluadas por el equipo y no se incorporaron automáticamente; se conservaron únicamente las que tenían justificación dentro del dominio del sistema.

FASE 5 — PLANIFICACIÓN DE LA SUSTENTACIÓN EN VIDEO

5.1 Enlace del Video de Sustentación

Enlace al Video de Exposición: <https://youtu.be/dy-CJhaxa8I>

5.2 Estructura y Planificación de Tiempos del Video (Máx. 20 min)

Minutos	Contenido Temático	Expositor(es) Responsable(s)	Soporte Visual / Acciones en Vivo
0 – 3 min	El sistema actual: qué hace, diagrama AS-IS y los tres puntos de dolor priorizados.	Todos / Integrador	Presentación del diagrama de clases AS-IS.
3 – 6 min	Hallazgos: propios, el refutado a la herramienta y contraste con hipótesis de lectura en frío.	Todos	Demostración de los archivos leídos y contraste de ideas.
6 – 12 min	Diagrama TO-BE en detalle: intervenciones de color, cada ADR, alternativas y principios aplicados.	Arq. de Dominio / Arq. de Dependencias	Recorrido por el diagrama TO-BE con código de color de SOLID.
12 – 17 min	Ejecución en vivo: demostración de comportamiento preservado y la solicitud de cambio implementada (métrica).	Ing. de Comportamiento / Integrador	Compilación y ejecución del Programa Principal en consola; mostrar test de caracterización.
17 – 20 min	Lecciones de IA: propuestas descartadas/no aplicadas, deudas técnicas conscientes y conclusiones finales.	Todos	Análisis crítico final del equipo frente a la IA.